Backend Engineering Task: Multi-Tenant Paystack Payment Gateway (Laravel)

Objective

Design and implement a **centralized Paystack payment gateway service** that can be used by **multiple Laravel applications**.
Each application must be able to:

- Initialize payments via **Paystack redirect URL**
- Receive and verify payments via **Paystack webhooks**
- Have **isolated credentials and callbacks**
- Reliably track and reconcile payment status

Scenario

Your company manages several Laravel-based products (e.g., e-commerce, subscriptions, event ticketing).
Rather than implementing Paystack separately in each app, you are tasked with building a **shared payment gateway module/service** that supports **multiple Laravel applications (tenants)** with minimal duplication.

Core Requirements

1. Multi-App (Tenant) Support

- Each Laravel app must have:
    - Its own **Paystack public & secret keys**
    - Its own **redirect (callback) URL**
    - Its own **webhook endpoint**
- The system must dynamically determine which app initiated a payment.

2. Payment Initialization (Redirect Flow)

Implement an endpoint that:

- Accepts payment initialization requests from a Laravel app
- Required fields:
    - `app_id`
    - `email`
    - `amount`
    - `currency`
    - `reference` (unique per transaction)
    - `metadata` (optional)
- Initializes Paystack payment and returns:
    - `authorization_url`
    - `reference`

3. Redirect Callback Handling

- Handle Paystack redirect responses
- Verify the transaction using Paystack's **verify endpoint**
- Update transaction status:
    - `pending`
    - `successful`
    - `failed`
- Redirect the user back to the originating Laravel app with the payment status

4. Webhook Processing

- Implement a secure webhook endpoint:

    - Validate Paystack signature (`x-paystack-signature`)
    - Process events such as:
        - `charge.success`
        - `charge.failed`
- Ensure:
    - Idempotency (same webhook not processed twice)
    - Webhook can update the payment even if the redirect was not completed

5. Transaction Management

Create a persistent transaction store with:

- `id`
- `app_id`
- `reference`
- `amount`
- `currency`
- `status`
- `gateway_response`
- `paid_at`
- `channel`
- `raw_payload`

6. Security & Reliability

- Validate webhook signatures

- Use environment-based secrets
- Prevent replay attacks
- Gracefully handle Paystack downtime
- Log all gateway interactions

Optional (Bonus) Features

- Support test & live Paystack environments per app
- Retry mechanism for failed webhook processing
- Queue webhook handling using Laravel Queues
- API rate limiting per app
- Support for partial payments or subscriptions

Technical Constraints

- Laravel 9+
- PHP 8+
- Paystack REST API
- RESTful architecture
- MySQL
- Use Laravel's HTTP Client

Deliverables

1. Database schema/migrations
2. API endpoints:
    - Initialize payment

    - Redirect callback
    - Webhook listener
3. Service class for Paystack integration
4. Configuration strategy for multi-app credentials
5. README explaining:
    - Setup
    - Environment variables
    - Payment flow
    - Security considerations

Evaluation Criteria

- Clean architecture & separation of concerns
- Proper Paystack API usage
- Secure webhook verification
- Multi-tenant design correctness
- Error handling & logging
- Code readability & documentation

Expected Payment Flow (High Level)

1. App → Gateway: Initialize payment
2. Gateway → Paystack: Create transaction
3. User → Paystack → Redirect URL
4. Gateway → Paystack: Verify transaction
5. Paystack → Gateway: Webhook confirmation
6. Gateway → App: Final payment status